

Industrial Internship Report on

“AutoParts Hub — Automotive Parts Management System”

Prepared by

Mumo

UCT Full Stack Internship Projects — UpSkill Campus

Executive Summary

This report provides details of the Industrial Internship offered by UpSkill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). The internship is centred on a project/problem statement provided by UCT, to be completed — including this report — within the internship period.

My project is AutoParts Hub, a full-stack web application for managing automotive parts. It gives automotive-parts businesses a centralised, user-friendly system for organising products, categories, vehicle information and stock records, replacing disconnected manual records with a searchable, maintainable application built on a React frontend, a Flask REST API backend, and a MySQL database.

[This section is a working draft — to be updated with a closing summary of outcomes and overall reflection once the remaining weeks of implementation and testing are complete.]

Preface

[Placeholder — to be completed once all internship weeks are finished. Suggested content:]

- A summary of the work completed across all weeks of the internship.
- Why an internship like this matters for career development — the shift from academic exercises to a real project brief, timeline and stakeholders.
- A brief recap of the AutoParts Hub problem statement and what the project sets out to solve.
- The opportunity provided by UpSkill Campus and UCT, and how the programme was structured week by week.
- Overall learnings and experience across the internship.
- Acknowledgements — supervisors, mentors and peers who provided support (to be named).
- A closing message to junior learners following the same programme.

Introduction

About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a primary focus on sustainability and return on investment (RoI). To develop its products and solutions, UCT leverages cutting-edge technologies including the Internet of Things (IoT), cybersecurity, cloud computing (AWS, Azure), machine learning, communication technologies (4G/5G/LoRaWAN), Java Full Stack, Python and front-end development.

UCT's product portfolio includes the UCT Insight IoT platform (built with a Java backend and a ReactJS frontend, supporting MySQL and various NoSQL databases, with device connectivity via MQTT, CoAP, HTTP, Modbus TCP and OPC UA), the Factory Watch smart factory platform for production and asset monitoring, LoRaWAN-based solutions for agritech, smart cities and utility metering, and predictive maintenance solutions combining embedded systems, industrial IoT and machine learning.

About UpSkill Campus (USC)

UpSkill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitates the smooth execution of the complete internship process. USC is a career development platform delivering personalised, self-paced upskilling alongside additional support services such as internships, projects, interaction with industry experts and career growth services, with the broader aim of upskilling 1 million learners over the next 5 years.

The IoT Academy

The IoT Academy is the EdTech division of UCT, running long-form executive certification programmes in collaboration with EICT Academy, IITK, IITR and IITG across multiple domains.

Objectives of this Internship Programme

- Gain practical experience of working in the industry.
- Solve real-world problems.
- Improve job prospects.
- Deepen understanding of the field and its applications.
- Support personal growth — better communication and problem-solving.

Reference

[1] — to be added

[2] — to be added

[3] — to be added

Glossary

Term	Acronym / Meaning
AutoParts Hub	Project name for the automotive parts management system
ORM	Object-Relational Mapping (used via Flask-SQLAlchemy)
API	Application Programming Interface
SKU	Stock Keeping Unit

Problem Statement

Automotive-parts businesses handle products, categories, vehicle information and stock records that need to be organised and accessed efficiently. Many such businesses currently rely on disconnected records — spreadsheets, paper logs or siloed systems — that make it difficult to search inventory, track stock levels accurately, or associate parts with the correct vehicles.

The AutoParts Hub project addresses this by providing a centralised full-stack web application through which categories, products and vehicle information can be managed through a single, user-friendly interface, rather than relying on disconnected records.

Existing and Proposed Solution

[Placeholder — to be expanded with research on existing tools once available. Suggested content:]

- Summary of existing solutions used by similar businesses (e.g. generic spreadsheet tracking, off-the-shelf inventory software) and their limitations — lack of vehicle-specific part associations, poor searchability, no centralised access across teams.
- AutoParts Hub's proposed value addition: a purpose-built, centralised web application with a relational data model connecting categories, products and vehicles, accessible through a modern React interface backed by a Flask REST API.

Code Submission (GitHub Link)

[Placeholder — update with the working link once the AutoPartsHub GitHub repository is finalised, e.g. <https://github.com/<username>/upskillcampus/blob/main/AutoPartsHub.py>]

Report Submission (GitHub Link)

[Placeholder — update with the working link once this report is uploaded, e.g. https://github.com/<username>/upskillcampus/blob/main/AutoPartsHub_Mumo_USC_UCT.pdf]

Proposed Design / Model

High Level Diagram

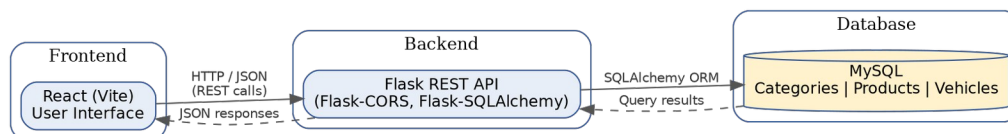


Figure 1: High-level architecture — React frontend, Flask REST API backend, and MySQL database.

Low Level Diagram

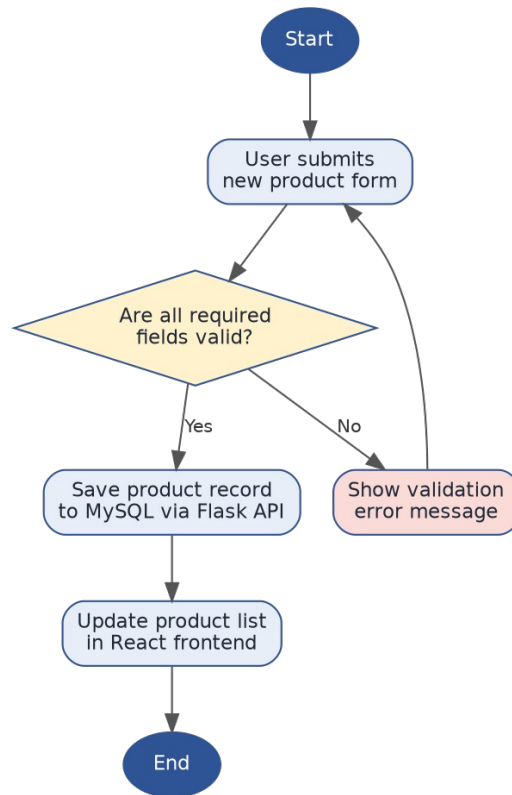


Figure 2: Add New Product flow — form submission, validation, save to database, and frontend list update.

Interfaces

The React frontend and Flask backend communicate over HTTP using JSON payloads through a REST API. Planned endpoints include product, category and vehicle CRUD operations (e.g. GET/POST /api/products, GET /api/categories, GET /api/vehicles), to be documented in full as implementation progresses.

[Placeholder — expand with the finalised endpoint list, request/response formats, and any additional block diagrams, data flow or state machine details as the API is implemented.]

Performance Test

[Placeholder — to be completed once implementation reaches a testable stage. This section should identify constraints relevant to AutoParts Hub — e.g. API response time, database query performance as product volume grows, data accuracy/validation, and concurrent access — and report how the design accounts for them.]

Test Plan / Test Cases

[To be added — e.g. test cases for product creation, validation failures, category/vehicle association, and stock updates.]

Test Procedure

[To be added — e.g. manual testing via the frontend, API testing via Postman/curl, and any automated test scripts.]

Performance Outcome

[To be added — recorded results once tests are run.]

My Learnings

Across the weeks completed so far, this internship has provided practical experience in:

- Planning a full-stack application before implementing its complete functionality — defining objectives, entities and architecture up front.
- Separating frontend and backend responsibilities clearly, and preparing a modular Python (Flask) and JavaScript (React/Vite) development environment.
- Considering the database entities and relationships required by a real-world system, and the importance of keeping the project structure modular so that functionality can be added without restructuring the whole application.
- Reinforcing core Java and object-oriented programming concepts (inheritance, polymorphism, abstraction, encapsulation) that underpin backend and Android development more broadly.
- Building a working understanding of full-stack architecture and HTML fundamentals, including how the MEAN stack demonstrates an end-to-end JavaScript approach as an alternative to the Python/React stack used in AutoParts Hub.

[To be expanded with career-relevant reflections once later weeks of implementation are complete.]

Future Work Scope

- Implement the full database schema in MySQL based on the categories/products/vehicles entity design.
- Build out the Flask REST API endpoints for CRUD operations on products, categories and vehicles.
- Connect the React frontend to the live API and replace placeholder UI with working data flows.
- Add authentication/authorisation if required for multi-user access.
- Implement stock-level tracking and low-stock alerts as a possible extension.
- Add automated tests and basic performance benchmarking once core features are complete.